Photo-to-Ukiyo-e Style Transfer using CycleGAN

Mini Project Report · ECSDI24402: Gen AI and Applications

1. Project Metadata and Identification

Field	Details
Project Title	Photo-to-Ukiyo-e Style Transfer using CycleGAN
Course Code and Title	ECSDI24402: Gen AI and Applications
Academic Program	Bachelor of Technology (B.Tech)
Branch and Semester	Computer Science and Engineering — Semester VII
Academic Year	2026 – 2027
Department	FEST (Faculty of Engineering Science and Technology)
Faculty Guide	Dr. Spoorthy Venkatesh
Submission Date	November 2026

2. Student Details

S. No.	Student Name	Enrollment No.	USN	Institutional Email
1	Nirav Maheta	230209	1AUA23BCS125	niravmaheta.cse23@adaniuni.ac.in

3. Executive Summary and Problem Statement

3.1 Background

A landscape photograph goes in. The intended output is the same place, drawn in the manner of a Japanese woodblock print: heavier outlines, flatter color, and the grain and palette of the prints in the training set. That print style is Ukiyo-e.

The public ukiyoe2photo collection is two unpaired folders, one of woodblock prints and one of landscape photographs. A print in the first folder is not the woodblock version of a photograph in the second. A paired image-to-image model needs a correct target for each input, and this collection does not provide one.

3.2 Problem Statement

The project trains a CycleGAN that translates photographs into Ukiyo-e while keeping the scene recognizable. Two generators learn the two directions at once. A photo is turned into a print and that print is turned back into a photo; the round trip is the stand-in for a paired answer. Two PatchGAN critics push each output to look as if it belongs in the target folder. The finished run is 20 epochs on a Tesla T4. Inference keeps only the photo-to-print network and runs it in the browser.

4. Project Objectives

- Train an unpaired photo-to-Ukiyo-e translator on ukiyoe2photo, direction BtoA (photograph to print).
- Use a 9-block ResNet generator and a 70×70 PatchGAN critic in each direction, with least-squares GAN loss, cycle loss, and identity loss.
- Finish 20 epochs: 10 at learning rate 0.0002, then 10 of linear decay to zero, with one resumable checkpoint.
- Judge the run from the eight loss curves and from eight held-out translations (photograph, print, reconstruction).
- Export the photo-to-print generator and serve it as a browser page. The server sends the file; the visitor's browser runs the network.

5. Mathematical and Theoretical Foundation

Domain A is prints and domain B is photographs. The option is BtoA. Before each loss, the two loaded batches are swapped, so inside the loss the photograph is real_A and the print is real_B. G_A reads the photograph and writes a print. G_B reads a print and writes a photograph. Each step draws one photograph x and one print y. They are not a pair. The batch size is 1, so a log line is one image.

The round trip and the identity check use L1 on pixel values in [−1, 1]. The cycle weight is 10, and the identity weight is half of that, so identity is multiplied by 5. The chart already includes these factors. A plotted cycle value near 1 means an average absolute round-trip error of about 0.1.

$cycle_A = 10 \cdot ||G_B(G_A(x)) - x||_1$ $cycle_B = 10 \cdot ||G_A(G_B(y)) - y||_1$

$idt_A = 5 \cdot ||G_A(y) - y||_1$ $idt_B = 5 \cdot ||G_B(x) - x||_1$

The critics use least-squares loss. A real image is asked to score 1 and a fake to score 0; the penalty is the squared distance from that target. Generators are trained while the critics are frozen, then the critics are trained while the generators are held still. A replay buffer of 50 recent fakes sometimes substitutes an older fake, so a critic does not forget mistakes the generator has already moved past. Identity loss asks G_A to leave a real print unchanged and G_B to leave a real photograph unchanged, which stops a translator from recoloring an image that is already in its target style.

6. System Architecture and Method

Both generators are a ResNet with nine residual blocks: reflection padding, a wide first convolution, two stride-2 downsamplings, nine residual blocks, two transposed convolutions, and a final tanh. Normalization is instance norm with no learned scale or shift. Dropout is off. Weights are drawn from a normal distribution with standard deviation 0.02. G_A and G_B each have 11.378 million parameters. Both critics are 70×70 PatchGANs (n_layers = 3) with 2.765 million parameters each. They score overlapping patches about 70 pixels across, so the generator is pushed on local texture: line, flat color, and grain.

Stage	What the run does
Data	One photo and one print per step. An epoch walks the larger folder: 6,287 images. The smaller folder is sampled so every step still has one image from each side.
Augment	Train: stretch to 286×286, random 256 crop, random left–right flip. Test: stretch to 256×256, no flip.
Translate	G_A makes a print from the photo. G_B makes a photo from the print. Each output is sent back through the other generator.
Losses	Least-squares GAN both ways, cycle L1 with weight 10, identity L1 with weight 5, replay buffer of 50.
Optimize	Adam, β1 = 0.5, β2 = 0.999. Learning rate 0.0002 for 10 epochs, then linear decay to 0 over 10 epochs.
Checkpoint	latest.pt is overwritten three times per epoch (steps 2,095, 4,191, and 6,287). It stores weights, optimizer state, the schedule, and the full loss history. A log line is kept every 100 steps.
Inference	The page loads G_A only, as ONNX (about 45 MB). The full checkpoint is about 340 MB and is not published. The browser stretches the photo to 256×256 and shows that square beside the print.

7. Technical Specifications and Toolchain

Category	Component	Role in this project
Training code	Single notebook	Generator, discriminator, dataset loader, and training step. Follows the CycleGAN design of pytorch-CycleGAN-and-pix2pix.
Framework	PyTorch	Networks, Adam, and the least-squares, cycle, and identity losses. Colab supplies PyTorch, torchvision, and Pillow.
Dataset	ukiyoe2photo	Unpaired prints and landscape photos, downloaded in the notebook. 6,287 steps per epoch.
Hardware	Tesla T4	GPU used for the saved 20-epoch run. The last epoch took 4,910 seconds.
Finished record	trained_fin.ipynb	Epoch 20 ending with learning rate 0, the loss chart, and eight test rows.
Inference	ONNX, browser	Frozen G_A. The page is a Vite app. A local script builds it and serves it at http://127.0.0.1:8000.
Hosting	Vercel	Production serves the built page from the master branch. The server does not translate the photograph.

8. Outcomes, Metrics, and Deliverables

8.1 Training that was completed

The saved run ends at epoch 20 of 20. The learning rate moves from 0.0000182 to 0, and latest.pt is written at the end of that epoch. The last epoch took 4,910 seconds on a Tesla T4. Opening the training notebook on this checkpoint does not continue training, because a learning rate of zero takes steps of size zero. About 1,257 points are logged (one every 100 steps). The left half of the chart is the ten epochs at 0.0002. The right half is the decay to zero.

The last logged step is epoch 20, iteration 125,700. It is one photograph and one print, not an epoch average. In the log after the BtoA swap, D_A is the print critic and G_A is the photo-to-print generator; D_B is the photo critic and G_B is the print-to-photo generator. cycle_A is photo, then print, then photo. cycle_B is print, then photo, then print. idt_A and idt_B are the identity terms. Cycle numbers include the factor of 10. Identity numbers include the factor of 5.

Term	Shape across 20 epochs	Last step (iter 125,700)
D_A	Stays jagged, mostly 0.05–0.3, spikes above 0.5	0.037
G_A	Stays jagged, mostly 0.2–1.0, spikes near 1.75	0.695
cycle_A	Starts near 8, falls in the first few epochs, then a noisy band around 1	0.822
idt_A	Starts near 2, then stays mostly below 1	0.356
D_B	Stays jagged, mostly 0.05–0.3, spikes above 0.5	0.105
G_B	Stays jagged, mostly 0.2–1.0, spikes near 1.75	0.534
cycle_B	Starts near 4.5, falls with cycle_A, then a noisy band around 1	0.570
idt_B	Starts near 2.5, then mostly below 1, with one later spike near 4	0.258

D_A, G_A, D_B, and G_B do not trend toward zero. For a least-squares GAN that is a balance: the critic keeps catching some fakes, and the generator keeps getting others accepted. A critic stuck at zero would have given up; a generator stuck at zero would no longer have a useful target. The noise is one image at a time.

cycle_A, cycle_B, idt_A, and idt_B are the content curves. The early drop means the round trip started keeping the input. The later flat band means that part of training had settled. A cycle value near 1 is an average absolute error of about 0.1 on the network's [−1, 1] scale.

8.2 Held-out translations

The notebook's last figure shows eight photographs the training loop did not use for those steps. Each row is the photograph (real_A), the print made by G_A (fake_B), and the photograph reconstructed by sending that print through G_B (rec_A). The scenes mostly survive in the third column, which matches the cycle curves settling near 1. The middle column, the actual product, is uneven. Some rows, such as the canal and the duck, read as painted. Others stay close to the photograph with a grain and a palette laid over it. The last landscape loses the hills. That spread is the quality this 20-epoch checkpoint reached. The figure is eight examples. It is not a score over the whole test set. FID, LPIPS, and SSIM were not computed.

8.3 Deliverables

- Training notebook and the finished-run notebook, with the loss chart and the eight test rows.
- Checkpoint latest.pt (all four networks and the loss history) and the browser copy of G_A, g_a.onnx, about 45 MB.
- A local page and the hosted page at https://ukiyoe-gan-ni3ravs-projects.vercel.app. The first translation downloads the model; later translations in the same visit reuse it.
- This report.

9. What This Run Adds, and Where It Stops

The collection has no paired photograph and print of the same place, so the round trip is what keeps the bridge, the hill, or the boat in the picture, and the print critic is what pushes the woodblock look. Publishing G_A alone means a visitor can restyle one photo without a GPU server: the hosted site only sends files.

The CycleGAN paper often trains for 200 epochs. This run uses 20. The cycle curves settle, and the reconstructions keep the layout, but several of the eight prints only partly take on the woodblock manner. That is the limit of this checkpoint, and it is visible in the test figure. The learning rate is now zero, so the same checkpoint is not a starting point for a longer run.

10. Practical Uses

- Restyling a landscape photograph into a woodblock-print manner for illustration or a personal print.
- A course demonstration of unpaired translation: two domains, no matched pairs, cycle loss as the content constraint, and a PatchGAN as the style constraint.
- A client-side demo. The 45 MB generator runs in the browser at 256×256, so the host does not need a GPU to answer a request.

11. Work Carried Out

Phase	Work	Result
1	Notebook with both generators, both critics, the loader, and the training step. Dataset downloaded in the notebook.	ukiyoe2photo, 6,287 images per epoch.
2	Train 20 epochs on a Tesla T4. Checkpoint overwritten three times an epoch so a dropped session can resume.	Epoch 20 ends at learning rate 0. Last epoch: 4,910 seconds.
3	Read the full loss history and the eight held-out rows.	Content curves fall and level off. Prints are uneven, as in §8.2.
4	Export G_A to ONNX and build the browser page.	256×256 photo beside the print. Model about 45 MB.
5	Publish the page. The build contains the site and G_A, not the 340 MB checkpoint.	Hosted page serves files only.

12. References

[1] Zhu, J.-Y., Park, T., Isola, P., and Efros, A. A. (2017). Unpaired image-to-image translation using cycle-consistent adversarial networks. In ICCV (pp. 2223–2232). IEEE.

[2] Isola, P., Zhu, J.-Y., Zhou, T., and Efros, A. A. (2017). Image-to-image translation with conditional adversarial networks. In CVPR (pp. 1125–1134). IEEE. The 70×70 PatchGAN used here is the critic from this line of work.

[3] Mao, X., Li, Q., Xie, H., Lau, R. Y. K., Wang, Z., and Smolley, S. P. (2017). Least squares generative adversarial networks. In ICCV (pp. 2794–2802). IEEE.

[4] Zhu, J.-Y., et al. pytorch-CycleGAN-and-pix2pix. https://github.com/junyanz/pytorch-CycleGAN-and-pix2pix. The notebook follows this implementation: resnet_9blocks, n_layers_D = 3, gan_mode lsgan, lambda 10, lambda_identity 0.5, pool size 50. The ukiyoe2photo set is the dataset shipped with that project.

[5] Ulyanov, D., Vedaldi, A., and Lempitsky, V. (2016). Instance normalization: The missing ingredient for fast stylization. ArXiv:1607.08022.